

User Similarity Recommender System

Authors: Ghina Al Shdaifat and Hamza Alshamy

Part 1: Customer Segmentation

Overview: This project addresses two core tasks related to user similarity on the MovieLens `ml-latest` dataset (33 million rows):

1. Identifying the top 100 most similar user pairs ("movie twins") based on shared movie-watching behavior.
 2. Validating whether these user pairs exhibit higher correlation in their actual rating behavior than 100 randomly selected user pairs.
-

Implementing MinHashLSH

1. Dataset and Preprocessing: We used the full `ml-latest/ratings.csv` file. To reduce noise and enhance signal quality, we filtered the dataset to:

- Only include users who rated at least **200** movies.
- Only include movies with at least **100** ratings.

This filtering step ensured that all users and movies in the similarity computation had a sufficient level of activity, thereby enhancing both computational efficiency and the relevance of the results. After filtering, the working dataset included **20,525,899 ratings** from **42,524 users**. Each user was represented by the set of `movieIds` they had rated, which captures their movie-watching "style," independent of rating values.

2. MinHash and Locality Sensitive Hashing (LSH): To efficiently scale user similarity comparisons, we employed the `datasketch` library to generate MinHash signatures for each user. We used **128 permutations**, which offered a balanced tradeoff between computational efficiency and estimation accuracy. In MinHash, the number of permutations directly affects the precision of the estimated Jaccard similarity: more permutations yield more accurate similarity estimates but also increase both memory usage and runtime. The value of 128 was selected because it is widely adopted in practice as a good compromise between speed and accuracy, particularly for large-scale datasets. Although using more permutations (e.g., 256 or more) could marginally improve estimation fidelity, doing so on a dataset with 33 million rows would significantly increase computational cost.

These MinHash signatures were then inserted into a Locality Sensitive Hashing (LSH) index (`MinHashLSH`) with a similarity threshold of **0.4**. This threshold determines the selectivity of the LSH index—that is, how aggressively it filters out dissimilar user pairs. We chose a threshold of 0.4 based on prior experimentation with the `ml-latest-small` dataset, where observed

Jaccard similarities among top user pairs rarely fell below 0.3. A higher threshold (e.g., 0.8) would return very few candidates, risking the omission of genuinely similar users. Conversely, a much lower threshold (e.g., 0.1) would yield an excessive number of pairs, increasing memory usage and runtime without added benefit. The 0.4 threshold thus provided a practical middle ground: it produced **10,390,581 candidate pairs** with meaningful movie overlap, while keeping the overall runtime tractable. On the full dataset, the total runtime was approximately **1329.47 seconds**.

3. Jaccard Distance Computation: For each candidate user pair identified by the LSH index, we computed the **exact Jaccard distance** between their movie-watching sets. Jaccard distance, defined as one minus the Jaccard similarity, quantifies dissimilarity between two sets. This step ensures that the final ranking of user pairs is based on **true set overlap**, rather than the approximate similarity used by MinHash for candidate generation. From the full set of candidate pairs, we selected the **top 100 user pairs with the lowest Jaccard distance**, representing those with the most overlapping movie histories. This hybrid approach combines the scalability of MinHashLSH with the accuracy of exact Jaccard evaluation, ensuring both computational efficiency and reliable similarity estimation for downstream validation.

4. Rating Correlation Validation: To evaluate whether the most similar user pairs also exhibited similar rating behavior, we computed **Spearman correlation coefficients** for each of the top 100 user pairs identified via Jaccard distance. A dictionary was first constructed to map each user to their rated movies and corresponding ratings (`user → {movieId: rating}`). For each pair, we computed the Spearman correlation over the set of movies they had both rated.

To establish a baseline, we also generated **100 random user pairs** from the same filtered user pool and computed their Spearman correlations using the same method. The average Spearman correlation from the top 100 pairs was then compared to that of the random pairs:

- **Mean Spearman correlation (top 100 pairs):** 0.8737
- **Mean Spearman correlation (random 100 pairs):** 0.1652

These results indicate that the “movie twins” identified via MinHash-based similarity not only watched similar sets of movies but also **rated them in highly consistent ways**, as reflected by their strong rank correlation.

We specifically chose **Spearman correlation** over Pearson because it is a **rank-based measure**, which is more appropriate for **ordinal data** such as 1–5 star movie ratings. Unlike Pearson, Spearman does not assume linear relationships or equal intervals between rating values. It is also more robust to outliers and better suited to capturing monotonic (but not necessarily linear) relationships, which are common in human rating behavior. Spearman correlation therefore provides a more meaningful assessment of whether users **ranked movies similarly**, even if their raw scores differed.

5. Output and Evaluation: The final output CSV (`/scratch/gha2009/top_100_movie_twins.csv`) contains the following columns:

- `userIdA, userIdB`
- `JaccardSimilarity, JaccardDistance`
- `SpearmanCorrelation`

Note: to view the full `top_100_movie_twins.csv` file, kindly find the physical file in: `scripts/top_100_movie_twins.csv`. Here is a sample of the first 10 rows of this file below:

userIdA	userIdB	Jaccard Similarity	Jaccard Distance	Spearman Correlation
137812	181530	1.0000	0.0000	0.9816
77647	294432	0.9981	0.0019	0.9999
19947	139527	0.9979	0.0021	0.4438
25084	86967	0.9883	0.0117	0.9971
30254	292994	0.9868	0.0132	0.9818
86967	294432	0.9857	0.0143	0.9972
25084	294432	0.9853	0.0147	0.9997
77647	86967	0.9839	0.0161	0.9971
25084	77647	0.9834	0.0166	0.9971

To further validate our correlation values, we additionally printed the following summary statistics to `stdout`:

- **Mean Spearman (top 100):** 0.8737
- **Mean Spearman (random 100):** 0.1652

These values affirm that users identified through MinHash-based similarity not only share similar movie-watching patterns but also tend to rate movies in a correlated fashion. Specifically, a value of 0.8737 for the top 100 similar user pairs suggests that the way users ranked or rated movies is strongly aligned—even if the absolute rating values differ. On the other hand, random user pairs tend to exhibit weak arguments, with a low spearman value of 0.1652, suggesting that such alignment is unlikely to occur by chance.

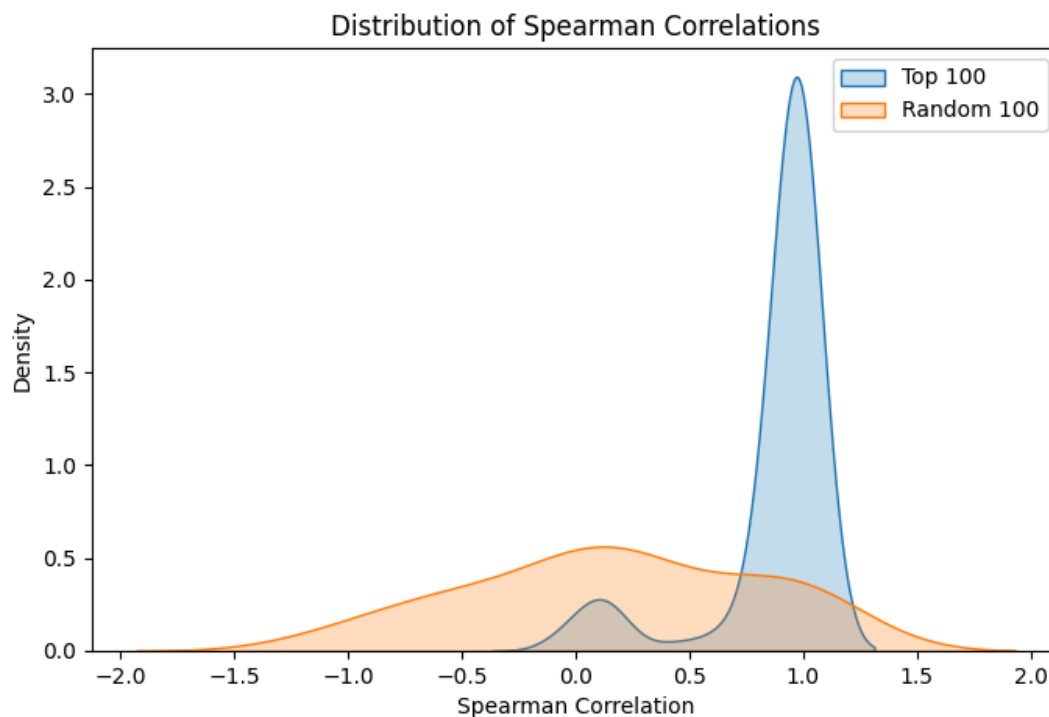
These findings reinforce that **watch history can effectively serve as a proxy for user preference**, and that MinHash-based segmentation identifies users who not only behave similarly but also share taste. This validates that the segmentation method identifies not just "active users," but users who **agree on what is good**.

To statistically validate this finding, we performed an **independent two-sample t-test** comparing the Spearman correlations of the top 100 pairs and 100 random user pairs. Our null hypothesis was that there is **no difference** in average Spearman correlation between the two

groups; the alternative hypothesis was that the **top 100 pairs have a significantly higher average correlation**.

- **T-statistic:** 6.2166
- **P-value:** 4.8517e-07

This result provides strong statistical evidence ($p < 0.00001$) that the top 100 most similar user pairs (based on watch history) also exhibit **significantly higher alignment in how they rate movies**.



Note: the code for this implementation (including the visualization) can be found in [scripts/correlation_analysis.py](#)

6) HPC Execution and Batch Processing

All code thus far in the project was run on NYU's High Performance Computing (HPC) cluster using SLURM-based batch jobs. A custom [.sh](#) file ([scripts/run_customer_segmentation_final.sh](#)) was used to specify job resources:

- **Runtime:** 4 hours
- **Memory:** 64 GB
- **CPUs:** 4 cores

The Python script (`scripts/customer_segmentation_final.py`) was submitted via: `sbatch run_customer_segmentation_final.sh`

Intermediate and final outputs (including CSV, visualizations, and logs) were written to the `/scratch/gha2009/` directory to avoid quota limitations in the home directory. The use of SLURM allowed us to process the full 33 million-row dataset efficiently while avoiding crashes or timeouts associated with interactive sessions. Physical local downloads have been made and committed to this repository for public viewing.

Part 2: Recommendation Phase

Popularity Baseline Model

Data Preprocessing and Partitioning: The data preprocessing and splitting pipeline was implemented in the `preprocess_data.py` script. The primary goal was to prepare the MovieLens ml-latest ratings data for model training, hyperparameter tuning, and final evaluation, while ensuring temporal integrity and preventing data leakage between splits.

We began by loading all relevant CSV files—`ratings.csv`, `tags.csv`, `movies.csv`, `genome-scores.csv`, and `genome-tags.csv`—from HDFS using explicit Spark schemas. The data was cleaned by removing missing values and duplicates across all tables using explicit schemas for each table.

Although we loaded and processed auxiliary files (tags and genome data), these were **not used in modeling**, but were included for potential downstream use. For example, the movies table was enriched with:

- A list of **genome tags** per movie
- The **median genome tag relevance score**
- A bag-of-genres representation using `CountVectorizer` (applied to the genre list)

These enriched features were saved in Parquet format (`movies_enriched`) for future access, but were **not merged with the ratings data** or used in the baseline model at this stage.

The core preprocessing step involved **splitting the ratings dataset chronologically on a per-user basis**:

- Ratings for each user were ranked by `timestamp_rating`
- The first **60%** of a user's ratings were assigned to the **training set**
- The next **20%** to the **validation set**
- The final **20%** to the **test set**

This time-aware splitting method ensures that models are always evaluated on future data relative to training and is robust against data leakage. The resulting splits—`ratings_train`, `ratings_validation`, `ratings_test`—were saved as separate CSVs in HDFS for downstream reuse.

Note: The full train/validation/test CSV splits exceed GitHub's 100 MB file limit and are therefore only stored on HDFS. For reproducibility and faster I/O in this project, we converted them to Parquet; those Parquet files live in `ALS_scripts/Parquet_files` in the repo (used in the following section).

Popularity-Based Baseline Model: To establish a simple baseline, we implemented a **non-personalized popularity model** in the `scripts/create_popular_movies.py` script. This model ranks movies based on their **average rating in the training set**, after filtering for movies with at least a minimum number of ratings (default: 100). The top K highest-rated movies (default: 30) are recommended to all users regardless of their individual preferences.

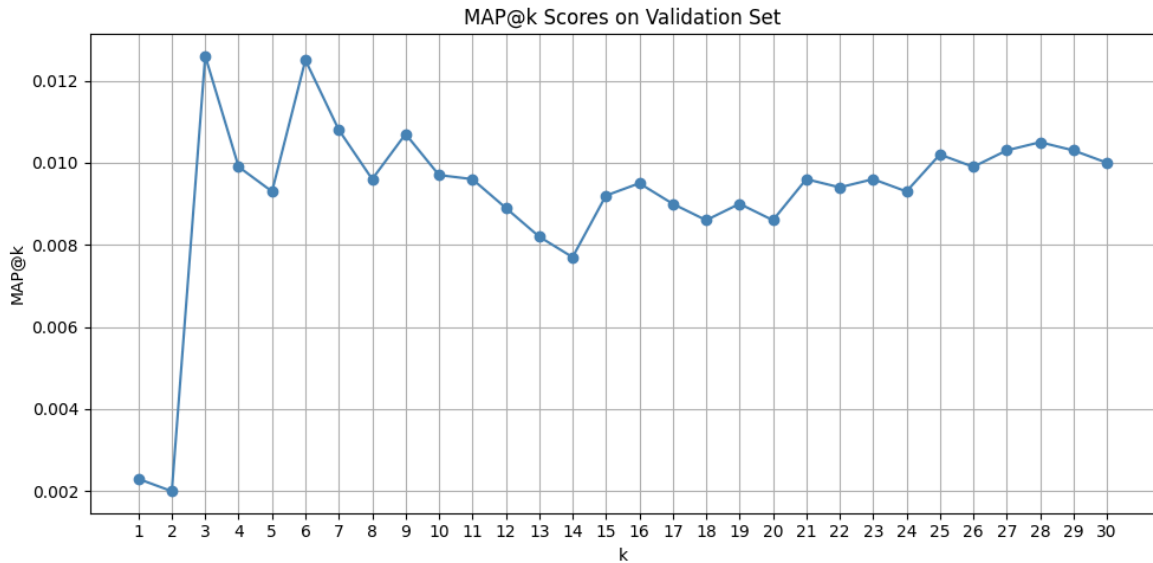
- **Evaluation of the Popularity-Based Baseline Model:** We evaluated this popularity-based model using the `evaluate_popularity_model.py` script. For each user in the validation and test sets, we computed **MAP@k** (**mean average precision at cutoff k**) by comparing the recommended movies against the user's actual highly-rated items (defined as ratings ≥ 3.0).

k	1	2	3	4	5	6	7	8	9	10
MAP@k	0.0023	0.0020	0.0126	0.0099	0.0093	0.0125	0.0108	0.0096	0.0107	0.0097

k	11	12	13	14	15	16	17	18	19	20
MAP@k	0.0096	0.0089	0.0082	0.0077	0.0092	0.0095	0.0090	0.0086	0.0090	0.0086

k	21	22	23	24	25	26	27	28	29	30
MAP@k	0.0096	0.0094	0.0096	0.0093	0.0102	0.0099	0.0103	0.0105	0.0103	0.0100

We computed MAP@k for k ranging from **1 to 100** on the validation set to observe how performance changes with cutoff. While **MAP@k peaked at k = 3 on validation**, we report test-set results at this cutoff for consistency with the baseline comparison.



Note: the code and actual png image of this visualization can be found in: [scripts/popularity_baseline_analysis.py](#), [scripts/map_at_k_plot.png](#)

After $K = 30$, our popularity-based model's $\text{MAP}@K$ approaches zero by $K = 100$. Beyond the first ~ 30 slots, the global popularity baseline has “used up” all of its truly relevant movies and is merely repeating the same top titles, which no longer match any user’s held-out interactions. Consequently, its $\text{MAP}@100$ falls to zero.

These results are exactly what we’d expect from a non-personalized baseline on a large, sparse dataset: limited absolute performance but a clear lower bound against which to benchmark more sophisticated models like ALS, which we explore next.

Latent Factor Model Using Alternating Least Squares (ALS)

Roadmap: For ALS, we wrote four scripts:

1. `convert_to_parquet.py`
2. `als_tuning.py` $\Rightarrow$ Tuning
3. `als_explicit.py` $\Rightarrow$ Running on best hyperparameters produced from tuning
4. `als_metrics.py` $\Rightarrow$ Evaluation metrics on the best model

Data Preprocessing and Partitioning: In order to ensure a fair comparison between our popularity-baseline and ALS latent-factor models, we adopted the same time-aware train/validation/test split for both approaches. Specifically, for each user in the MovieLens dataset, we first sorted all of their ratings by timestamp. And, second, assigned the first 60 % of ratings to the training set, the next 20 % to the validation set, and the final 20 % to the test set.

Similar to the popularity-based model, this per-user chronological split guarantees that every model is evaluated on “future” interactions, avoiding any leakage of information from later ratings into model training. To accelerate both hyperparameter tuning and final model training, we converted CSV files into parquet format. This format adopts columnar storage, efficient and compression. The three parquet files can be accessed in [ALS_scripts/Parquet_files](#).

Hyperparameter Tuning (Rank and Regularization):

In order to run the ALS model, we had to choose the values of the two parameters. The first parameter, **rank** (r), is the number of latent factors for users and items. Higher rank would allow the model to capture more nuanced preferences, but increase both computational cost and risk of over-fitting. The second parameter, **regularization** (λ), is the weight on the penalty term. A larger λ here reduces over-fitting by shrinking factor magnitudes, at the expense of potentially under-fitting.

Generally, our tuning procedure consisted of a grid search over several combinations of (r , λ) on a 40 % random subsample of our train/validation split (to speed up iterations), holding the number of ALS iterations at 10 and evaluating with MAP@100 on the validation set.

Our **first iteration** consisted of the following parameters, which produced very weak results:

Rank	Reg Param	Max Iter	MAP@100
10	0.01	10	0.0000
10	0.10	10	0.0000
10	1.00	10	0.0000

The uniformly zero MAP@100 for our first grid search reflects **two key issues** at the scale of such a large, sparse dataset. **First**, a latent dimension of only 10 is too low to capture the diversity of user–item interactions, so the model cannot place relevant items in users’ top-100 lists. **Second**, by fixing $\text{maxIter} = 10$, the solver often stops before factors have meaningfully converged. We deliberately kept iterations at 10 throughout tuning for computational efficiency and to limit total runtime, however, this efficiency-driven shortcut means many (r , λ) pairs never exit their “cold start”.

Our **second iteration** consisted of the following:

Rank	Reg Param	Max Iter	MAP@100
40	0.05	10	0.0010
40	0.10	10	0.0003
40	0.20	10	0.0000
50	0.05	10	0.0014
50	0.10	10	0.0004
50	0.20	10	0.0000
60	0.05	10	0.0019
Best params: rank=60, regParam=0.05 (MAP@100=0.0019)			

By raising the latent factor dimensionality to 40 and 60 (still with maxIter = 10) and testing light regularization, we finally saw MAP@100 climb above zero, peaking at 0.0019 for rank = 60 and $\lambda = 0.05$. These results confirm that a rank of only 10 was too low to surface any relevant items, and that modest shrinkage is preferable—stronger regularization ($\lambda \geq 0.10$) drove performance back toward zero.

Given the conclusions made above, our **third** iteration increased rank and kept $\lambda \geq 0.10$:

Rank	Reg Param	Max Iter	MAP@100
50	0.02	10	0.0005
50	0.05	10	0.0013
50	0.10	10	0.0005
60	0.02	10	0.0010
60	0.05	10	0.0018
60	0.10	10	0.0005
70	0.02	10	0.0011
70	0.05	10	0.0020
70	0.10	10	0.0006
Best params: rank=70, regParam=0.05 (MAP@100=0.0020)			

Two observations: **First**, the regularization parameter $\lambda = 0.05$ yields the highest MAP@100 at every tested rank. **Second**, when λ is fixed at 0.05, MAP@100 consistently increases with rank. Based on these findings, we fixed $\lambda = 0.05$ and continued to raise the rank, leading to our **fourth and final iteration** of tuning:

Rank	Reg Param	Max Iter	MAP@100
80	0.05	10	0.0022
90	0.05	10	0.0026
100	0.05	10	0.0024
Best params: rank=90, regParam=0.05 (MAP@100=0.0026)			

Based on the results above, we ran the full model on the best parameters (rank = 90, and $\lambda = 0.05$) and produced the following **evaluation metrics**:

Rank	Reg Param	MAP@100	NDCG	Precision	MRR
90	0.05	0.0030	0.0040	0.0011	0.0052

Params used: rank = 90, regParam = 0.05

Our final ALS model achieves MAP@100 = 0.0030 (i.e., only 0.3% of the top 100 recommendations are relevant on average), NDCG@100 = 0.0040, and MRR = 0.0052, confirming that its absolute ranking power is weak. Nevertheless, ALS still substantially outperforms the non-personalized popularity baseline (MAP@100 $\approx$ 0.0000). Although both values are small, the fact that ALS surfaces any held-out items in its top 100 demonstrates its ability to capture long-tail, personalized recommendations. Recasting ratings as implicit feedback, treating observed interactions as positives and sampling negatives, could generate denser training signals and improve relevant-item retrieval.